

BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible because there are simply too many possible moves to remember. A table-driven agent works by looking up a massive list that tells it exactly what to do in every possible situation. In a game like Chess, there are more possible ways the board can look than there are atoms in the universe. There is no computer big enough to store a list of that size.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Condition Action rules are just basic "IF-THEN" decisions that the agent makes. The Condition is the agent checking what is happening around it right at this exact moment (like asking, "Is there a wall in front of me?"). The Action is the physical move it decides to make based on that information (like deciding to "turn left").

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The agent got stuck because it is near-sighted and has zero memory. Because it can only see one step ahead, it didn't realize it walked into a dead-end. When it hit a wall, it turned, took a step, and hit another wall. Because it has no memory whatsoever, it instantly forgot about the first wall. It treats every single step like a brand new experience, so it just keeps repeating the exact same mistake forever.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The upgraded agent escapes because it draws a map in its head as it moves.

Transition Model: The agent keeps track of its own turns and steps so it knows which direction it is facing at all times.
Sensor Model / Memory: The agent remembers every single square it has already walked on. When it gets trapped in a corner, it doesn't just guess which way to go. It checks its memory, realizes it has already been to the left, and decides to turn right

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING